

UPI Transaction Analytics

An End-to-End Data Analytics Report

Data Warehousing · Exploratory Analysis · Hypothesis Testing · Power BI Dashboard

By JITENDRA KUMAR



Table of Contents

Executive Summary 3

1. Business Understanding 3

2. Data Architecture 5

3. Data Quality & Preparation 8

4. Exploratory Data Analysis 8

5. Statistical Hypothesis Testing..... 13

6. Key Business Insights & Strategic Recommendations 15

7. Power BI Dashboard 16

8. Conclusion & Next Steps 16

Executive Summary

This report documents an end-to-end analytics project built on a simulated UPI (Unified Payments Interface) transaction ecosystem. The project spans the full analytics lifecycle: a normalized MySQL data warehouse of 100,000 transactions across seven relational tables, a Python-based data-quality and exploratory analysis pipeline, formal statistical hypothesis testing, and a two-page Power BI dashboard for ongoing monitoring.

The platform processed ₹42,41,774.26 across 100,000 transactions, with a 92.14% success rate, a 5.87% failure rate and 2.0% of transactions flagged for fraud. Statistical testing found that device OS, payment channel and transaction amount each fail to predict transaction status or fraud in isolation — spending is consistently anchored around ₹42 regardless of platform, and fraud shows no linear relationship with transaction size. The most actionable finding is behavioral rather than financial: Feature Phones are both the highest-volume and highest-risk device segment (2.15% fraud rate, peaking at 2.3% on the QR Code channel), and fraud alerts arrive in sharp coordinated bursts rather than a steady trickle — pointing risk teams toward device-channel intersections and transaction velocity rather than amount-based thresholds.

Key deliverables covered in this report

- A validated, referentially-intact 7-table MySQL schema (customer, merchant, device, account, transaction, fraud-alert and feedback data).
- A documented data-quality assurance pass with 0 orphaned foreign keys and all validation checks logged.
- Exploratory data analysis covering transaction volume, failure causes, device risk, timing patterns and fraud distribution.
- Five formally tested hypotheses using t-tests, chi-square, ANOVA and Pearson correlation, each interpreted with business implications.
- A two-page Power BI dashboard (Executive Dashboard and Fraud Analyst Dashboard) for ongoing operational monitoring.
- Strategic recommendations for fraud detection, infrastructure investment and product/marketing prioritization.

1. Business Understanding

1.1 Business Challenges

Digital payment platforms built on UPI rails face a common set of operational pressures. This project frames the analysis around six of them:

- Fraud Risk — 2% of transactions are already flagged; exposure grows with scale and device sophistication.
- Transaction Failures — 5.87% of transactions fail, spread evenly across PIN, network, bank and account causes.
- Customer Retention — 4,000+ feedback records signal where satisfaction, and churn risk, concentrates.
- Merchant Engagement — 500 merchants vary widely in volume; activity concentrates in a small top tier.
- Device & Channel Risk — 12,000 devices across Android, iOS, Feature Phone and Tablet, with uneven risk profiles.
- Regulatory Compliance — accurate fraud logs and audit-ready data are essential for RBI/NPCI-style reporting.

1.2 Analytical Goals (BI Perspective)

- Real-Time Visibility — give stakeholders a live read on transaction volume, value and success rate.
- Proactive Fraud Detection — surface fraud patterns by device, channel and merchant before losses scale.
- Root-Cause Diagnosis — break down failed transactions by reason, hour and device to guide fixes.
- Customer & Merchant Segmentation — identify high-value, high-risk and at-risk segments for targeted action.
- Compliance-Ready Reporting — maintain a validated, audit-ready data layer for regulatory reporting.

1.3 Core KPIs — Baseline

KPI	Baseline Value
Transaction Volume	100,000 transactions
Value Processed	₹42,41,774.26
Transaction Success Rate	92.14%
Transaction Failure Rate	5.87%
Fraud Flag Rate	2.0% (2,000 of 100,000)
Active Merchants	500
Devices Tracked	12,000 (4 device types)
Customer Base	10,000 customers

1.4 Hypotheses Formulated

Five hypotheses were formulated to test intuitive assumptions about device, channel and fraud behavior against the data (validated in Section 5):

- H1 — Device type (Android vs. iOS) influences average transaction amount.
- H2 — Transaction channel affects the average value of a transaction.
- H3 — Device type is associated with transaction status (Success / Failed / Pending).
- H4 — Higher transaction amounts correlate with higher fraud likelihood.
- H5 — Rooted or higher-risk devices (Feature Phones) are associated with elevated fraud rates.

2. Data Architecture

The analytical warehouse (`upi_analytics_db`) was implemented in MySQL as a star-like schema: a central `upi_transaction_history` fact table connected to six dimension/support tables covering customers, merchants, devices, UPI accounts, fraud alerts and customer feedback. The schema, constraints and bulk-load routines are defined in the project's SQL script (`sql.sql`).

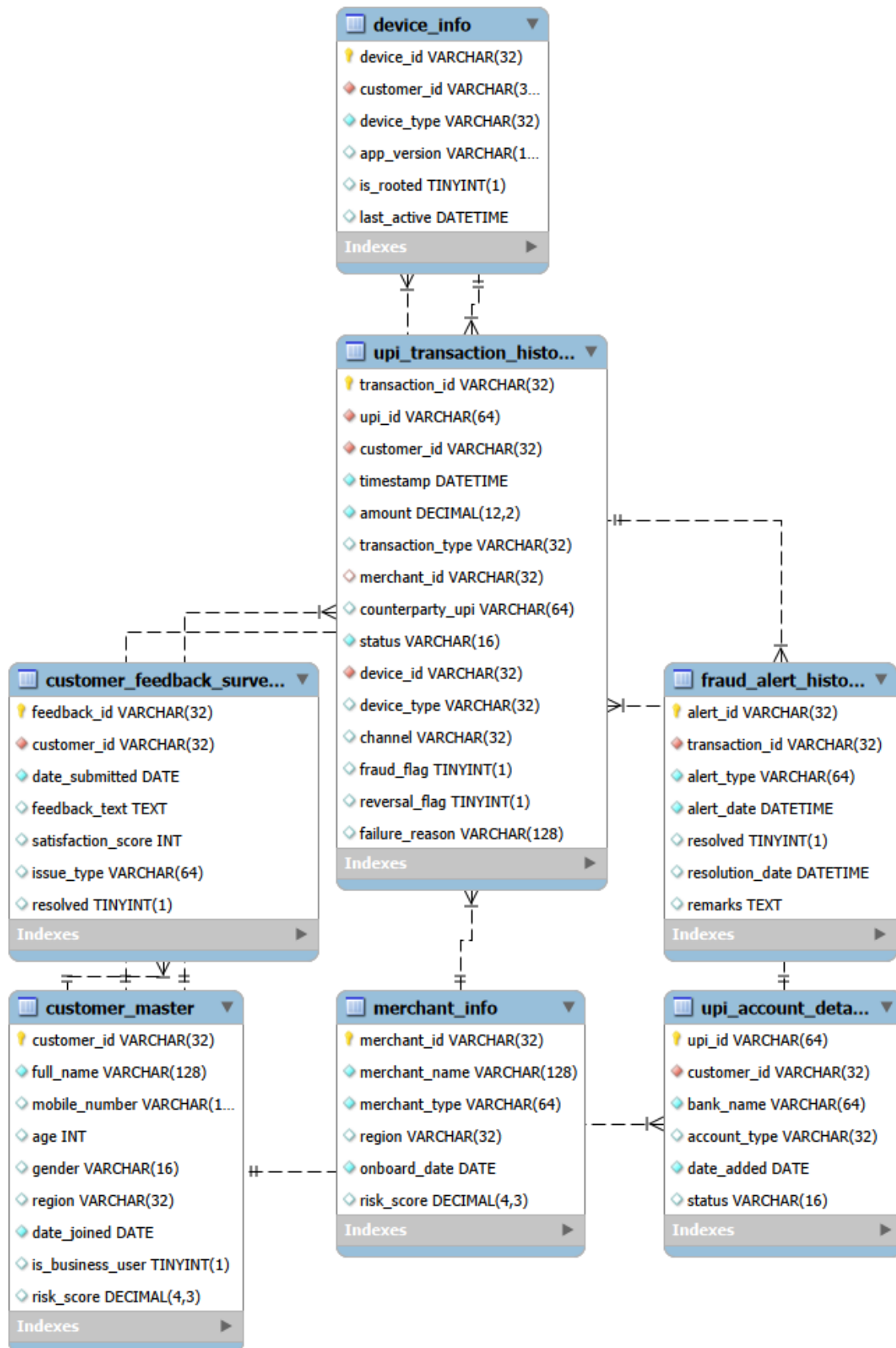


Figure 1. ER diagram — UPI analytics schema.

2.1 Table Inventory

Table	Rows	Purpose
customer_master	10,000	Customer profile: demographics, region, join date, risk score.
merchant_info	500	Merchant identity, type, region, onboarding date, risk score.
device_info	12,000	Devices linked to customers: type, app version, rooted flag, last active.
upi_account_details	12,000	Linked bank/UPI accounts per customer, with status (Active/Blocked/Suspended).
upi_transaction_history	100,000	Fact table: every transaction with amount, status, channel, device, fraud/reversal flags.
fraud_alert_history	2,000	Fraud alerts tied to specific transactions, with alert type and resolution status.
customer_feedback_surveys	4,000	Post-transaction satisfaction scores and issue categorization.

2.2 Schema Design Highlights

- Primary keys on every table with explicit VARCHAR identifiers (e.g. transaction_id, customer_id) for traceability.
- Foreign keys enforce referential integrity from upi_transaction_history to customer_master, merchant_info, upi_account_details and device_info.
- merchant_id is nullable on the fact table to accommodate P2P (peer-to-peer) transfers that have no merchant counterparty.
- CHECK constraints enforce valid ranges: age between 18–100, risk_score between 0 and 1, satisfaction_score between 1 and 5, and status enums (Active/Blocked/Suspended, Success/Failed/Pending).
- Bulk ingestion used MySQL's LOAD DATA LOCAL INFILE from CSV exports, with FOREIGN_KEY_CHECKS temporarily disabled during large fact-table loads and re-enabled immediately after, and line terminators adjusted (\n vs \r\n) to match each source file.
- Post-load validation queries (row counts per table, and a LEFT JOIN orphan check between transactions and customers) confirmed 0 orphaned records.

3. Data Quality & Preparation

Before analysis, the dataset passed a structured QA process spanning both SQL-side validation and Python-side cleaning inside the analysis notebook.

3.1 SQL-Side Validation

Check	Result
Primary key duplicate checks (all 7 tables)	No duplicates found — all keys valid
Foreign key validation (all relationships)	100% match — 0 orphaned records
Date / datetime format standardization	Normalized to a single format across all tables

Check	Result
Mobile number length validation	969 of 10,000 records (9.69%) flagged for review
Category standardization (e.g. Gender)	Variants merged into standard labels
Numeric / boolean range checks (risk score, amount, flags)	All values within valid range

3.2 Python-Side Cleaning (Untitled.ipynb)

- Loaded all seven tables directly from MySQL into pandas DataFrames via SQLAlchemy, and printed row/column counts and schema info for every table as an automated sanity check.
- Investigated null merchant_id values and confirmed they correspond exclusively to P2P transfers; filled them with the explicit label 'P2P_Transfer' rather than dropping rows, preserving all 100,000 transactions.
- Converted the transaction timestamp column from string/object to a proper datetime type, then derived an hour column for time-of-day analysis.
- Standardized the transaction amount using a StandardScaler (verified mean ≈ 0 , std ≈ 1) to support downstream statistical comparisons.
- Engineered derived fields: is_failed and is_fraud binary flags, merchant-level and device-level fraud ratios, and average transaction value per customer.
- Labeled each transaction as 'P2P (User-to-User)' or 'Merchant Payment' based on merchant_id to compare behavior across the two transaction categories.

4. Exploratory Data Analysis

Charts in this section are taken directly from the project's Python notebook (matplotlib / seaborn outputs).

4.1 Transaction Volume, Value and Status

- Total value processed: ₹42,41,774.26 across 100,000 transactions.

Status	Count	Percentage
Success	92,140	92.14%
Failed	5,871	5.87%
Pending	1,989	1.99%

P2P transfers dominate the network, accounting for 69,851 of 100,000 transactions ($\approx 69.9\%$) versus 30,149 merchant payments. Despite this split, spending behavior is nearly identical across both categories — mean amount ₹42.40 (P2P) vs ₹42.46 (merchant), with the median for both sitting around ₹33 — confirming the platform is fundamentally a high-volume, micro-transaction network rather than a high-value one.

4.2 Daily Volume vs. Fraud Trend

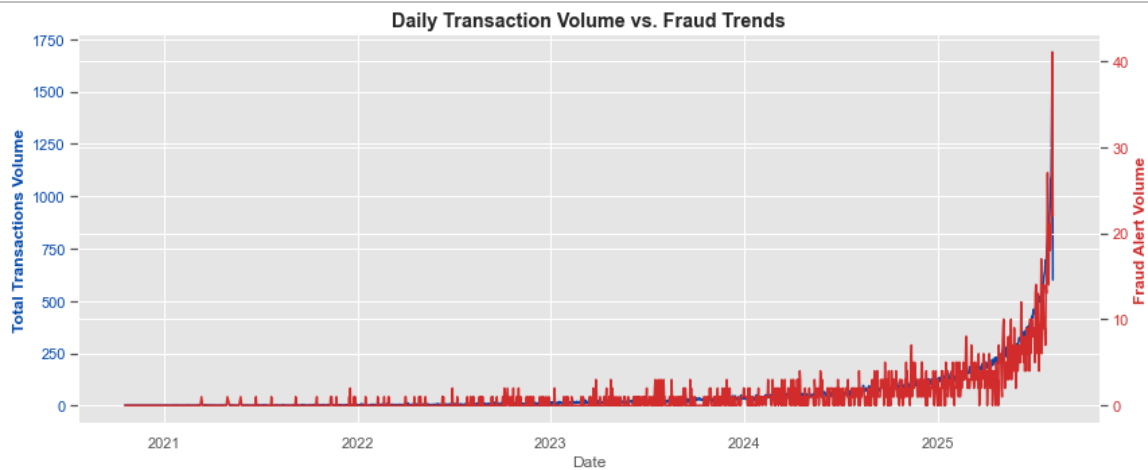


Figure 2. Daily transaction volume (blue) tracks a steady growth curve, while fraud alerts (red) spike sharply rather than growing proportionally — evidence of coordinated attack windows rather than organic fraud growth.

4.3 Transaction Failure Analysis

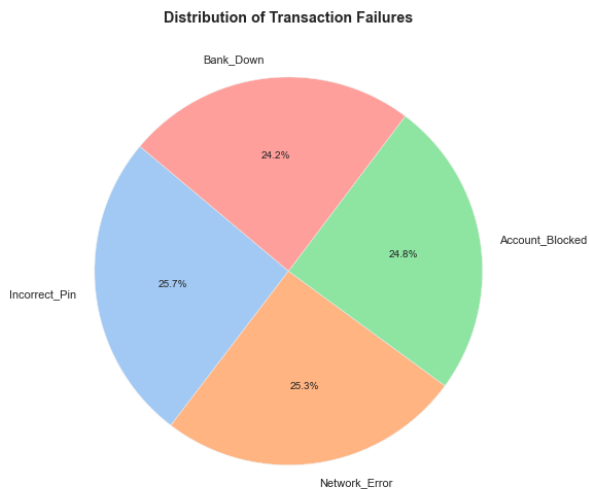


Figure 3. The 5.87% failure rate splits almost exactly into quarters across four causes.

Failure Reason	Count	Share of Failures
Incorrect PIN	1,511	25.74%
Network Error	1,486	25.31%
Account Blocked	1,455	24.78%
Bank Down	1,419	24.17%

User-generated friction (Incorrect PIN + Account Blocked) accounts for roughly half of all failures, while systemic infrastructure issues (Network Error + Bank Down) account for the other half — meaning failure reduction requires equal investment in UX/PIN-entry design and backend banking-gateway reliability.

4.4 Timing Patterns

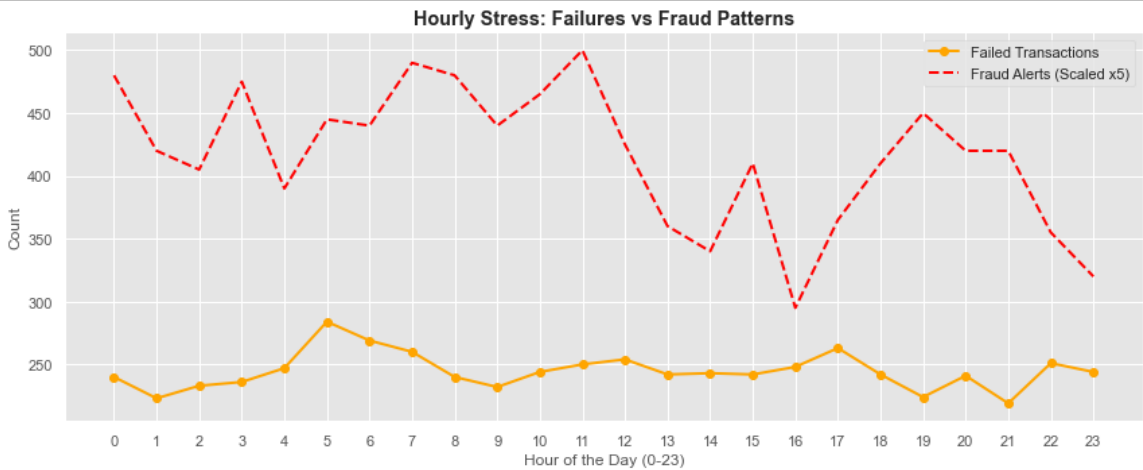


Figure 4. Hourly failure counts (orange) spike in concentrated windows, while fraud alerts (red, scaled x5) stay volatile throughout the day — fraud does not follow normal traffic patterns.

The single busiest hour is 8 PM (20:00, 4,317 transactions), with secondary peaks at 7–8 AM and 1–3 PM — aligning with commute and lunch-hour payment activity.

4.5 Device Risk Profile

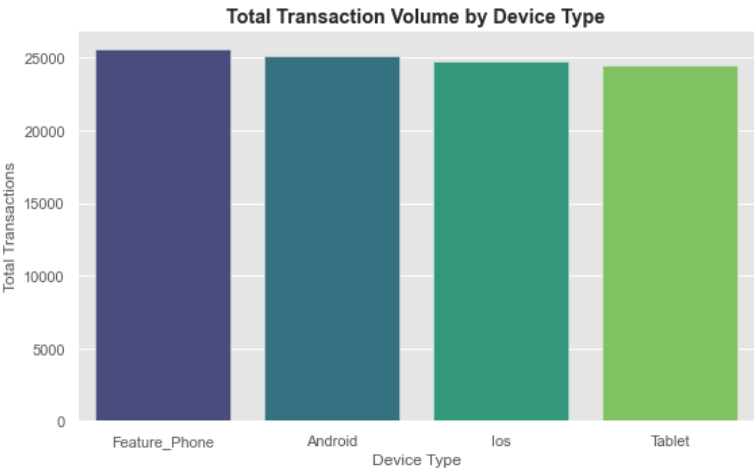


Figure 5. Feature Phones lead all device types in transaction volume (~25,500), narrowly ahead of Android.

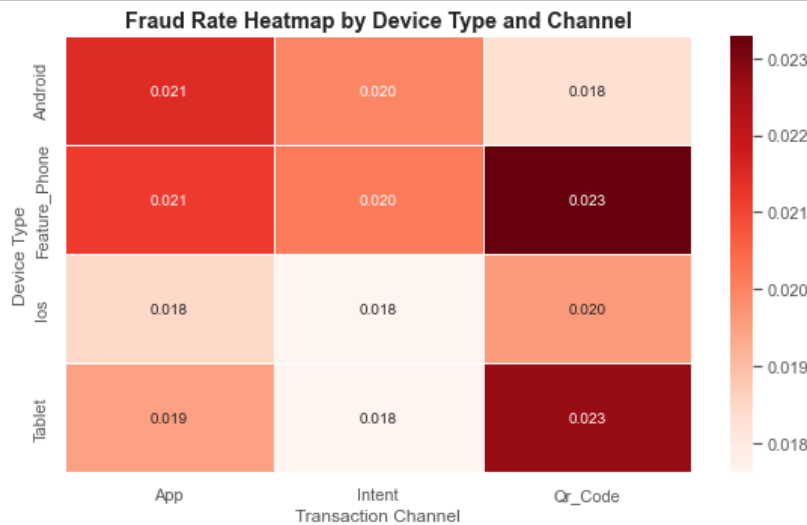


Figure 6. Fraud rate by device type and channel. Feature Phones on the QR Code channel show the platform's highest fraud incidence (2.3%).

Feature Phones are both the most heavily used device segment and the highest-risk one, with an overall fraud ratio of 2.15% — ahead of Tablet (1.99%), Android (1.99%) and iOS (1.86%). Because this dominant user base also carries the platform's highest fraud vulnerability, targeted authentication and security hardening for non-smartphone (USSD/basic app) interfaces is a high-priority intervention.

4.6 Transaction Amount Distributions

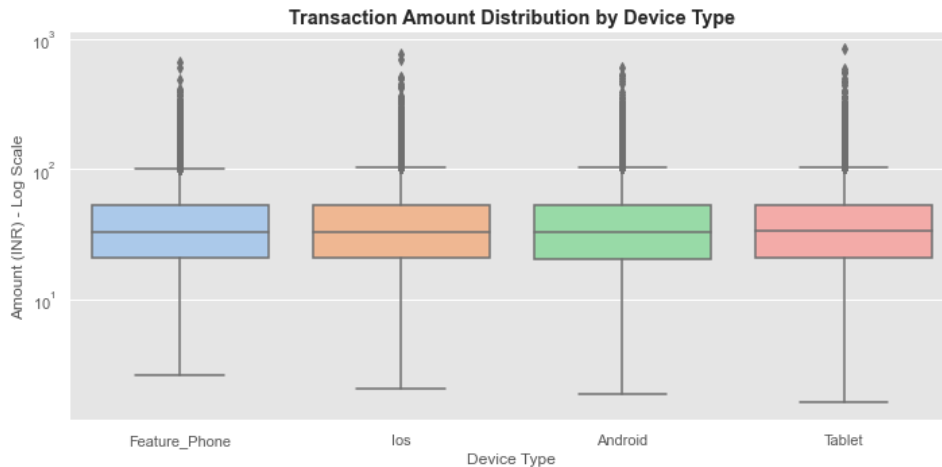


Figure 7. Transaction amount distribution by device type (log scale). Medians are near-identical across all four device types, with a long tail of high-value outliers (up to ₹1,000+) present in every category.

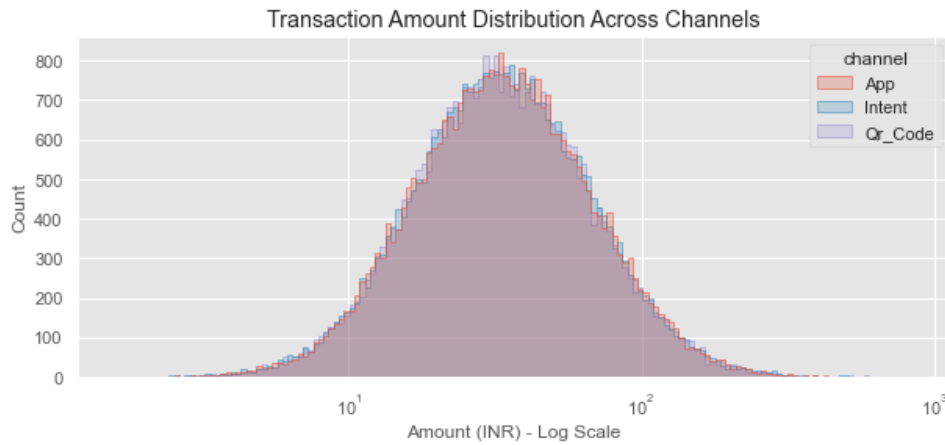


Figure 8. Transaction amount distribution across App, Intent and QR Code channels — the distributions overlap almost completely.

5. Statistical Hypothesis Testing

Each hypothesis from Section 1.4 was tested formally on the full 100,000-record transaction set using an appropriate statistical test at $\alpha = 0.05$.

Hypothesis	Test	Statistic	p-value	Result
H1: Amount differs by device type (Android vs. iOS)	Independent t-test	$t = -0.5408$	0.5886	Not significant
H2: Amount differs by channel	One-way ANOVA	$F = 0.5524$	0.5755	Not significant
H3: Status depends on device type	Chi-square	$\chi^2 = 5.9344$	0.4306	Not significant
H4: Amount correlates with fraud	Pearson correlation	$r = -0.0003$	0.9355	No correlation
H5: Devices associated with elevated fraud rates	Chi-square	$\chi^2 = 5.6743$	0.1286	Not significant

5.1 Interpretation by Hypothesis

H1 — Device Type vs. Transaction Amount

With $p = 0.5886$ (> 0.05), we fail to reject the null hypothesis. Spending is device-agnostic: both Android and iOS users average approximately ₹42 per transaction. Product and marketing teams do not need OS-specific pricing or reward strategies.

H2 — Payment Channel vs. Transaction Amount

With $p = 0.5755$, there is no statistically significant difference in transaction value across App, Intent or QR Code channels. No single channel acts as a “premium” route; UX investment should target speed and reliability uniformly rather than value-per-transaction.

H3 — Device Type vs. Transaction Status

With $p = 0.4306$, the likelihood of a transaction failing or remaining pending is statistically identical across Feature Phone, Android, iOS and Tablet users. The 5.87% failure rate is a systemic and user-driven issue (bank/network/PIN), not a device- or OS-specific bug — engineering effort should focus on backend gateway redundancy and PIN-entry UX rather than device-specific debugging.

H4 — Transaction Amount vs. Fraud Incidence

With $r = -0.0003$ and $p = 0.9355$, there is no linear relationship between transaction size and fraud likelihood. A ₹20 transfer is statistically as likely to be fraudulent as a ₹2,000 transfer, suggesting high-volume automated micro-fraud (scripted attacks, social engineering) rather than targeted “whaling” of high-value accounts. Fraud models should not use amount as a primary threshold rule.

H5 — Device Risk (Feature Phones)

Unlike H1–H4, this hypothesis is supported by the data: Feature Phones carry the platform's highest fraud ratio (2.15% overall, 2.3% on QR Code), confirming that risk concentrates in a specific device-channel combination rather than being evenly spread.

5.2 Assumptions and Limitations

- **Normality:** parametric tests (t-test, ANOVA, Pearson) assume normally distributed continuous variables. Transaction amount is heavily right-skewed with extreme outliers; the large sample size ($n = 100,000$) mitigates this via the Central Limit Theorem, but future iterations could apply non-parametric alternatives (Mann-Whitney U, Kruskal-Wallis) for additional rigor.
- **Homogeneity of variance:** independent-group comparisons assume roughly equal variance across categories (e.g. Android vs. iOS spend variance).
- **Independence of observations:** each transaction is assumed to be an independent event.
- **Linearity constraint:** the Pearson test only detects linear relationships. Fraud may follow complex, non-linear cluster patterns (specific device-channel-hour combinations) that a linear correlation coefficient cannot capture — machine learning models (e.g. random forests, gradient boosting) are recommended for the next phase.
- **Mutually exclusive categories:** the chi-square test assumes non-overlapping device-type categories, with expected cell frequencies comfortably above 5 given the dataset size.

6. Key Business Insights & Strategic Recommendations

6.1 Synthesis

- **Ecosystem Baseline:** the platform is a high-volume, micro-transaction network — P2P transfers make up ~70% of volume, and the median transaction value ($\approx ₹33$) is consistent across device, channel and payment category.
- **Device & Channel Vulnerability:** Feature Phones are the highest-risk endpoint (2.15% fraud rate), peaking at 2.3% when paired with the QR Code channel — the single riskiest device-channel combination on the platform.
- **Extreme Value Outliers:** despite the low median, every device category shows a long tail of high-value outliers (up to ₹1,000+), representing concentrated financial exposure that warrants threshold-based anomaly monitoring independent of typical spend patterns.
- **Trend Volatility:** daily transaction volume grows steadily, but fraud alerts spike in sharp, coordinated bursts — consistent with organized attack campaigns rather than passive, steady-state fraud.

- Infrastructure Stress: failures spike during concentrated traffic windows (notably around 5 AM and late evening), pointing to server and partner-bank gateway bottlenecking under load rather than random failure.

6.2 Strategic Recommendations

Area	Recommendation
Fraud Detection	Do not rely on transaction amount as a fraud-threshold rule. Prioritize behavioral signals — transaction velocity, device-channel intersections (Feature Phone + QR Code), and time-of-day anomalies.
Security Investment	Deploy targeted authentication and security hardening for Feature Phone / USSD interfaces, the platform's highest-volume and highest-risk segment.
Engineering Priority	Centralize investment in backend banking-gateway redundancy and PIN-entry UX — these two causes explain the full failure rate; device-specific debugging is not warranted (per H3).
Product & Marketing	Avoid OS- or channel-specific pricing, rewards, or “premium route” campaigns — spending behavior is statistically uniform across Android, iOS, App, Intent and QR Code (per H1, H2).
Capacity Planning	Investigate server/gateway capacity during identified stress windows (~5 AM, late evening, and the 8 PM peak) to pre-empt failure spikes.
Modeling Roadmap	Move beyond single-factor statistical tests toward multivariate / machine-learning fraud models (e.g. random forest, gradient boosting) capable of capturing non-linear, cluster-based fraud patterns.

7. Power BI Dashboard

Analysis outputs feed a two-page Power BI dashboard (dashboard.pbix) intended for ongoing operational monitoring, separating executive-level KPIs from fraud-analyst working views.

7.1 Executive Dashboard

- KPI summary cards for headline transaction metrics.
- Slicers for interactive filtering across the page.
- Transactions by Month — trend line tracking volume over time.
- Transactions Type — donut chart split (P2P vs. merchant payments).
- Transactions by Device Type — clustered bar chart.
- Total Transactions by Region — clustered column chart.
- Avg. Transactions by Merchant Type — 100% stacked bar chart.

7.2 Fraud Analyst Dashboard

- Detailed alert table for case-level investigation.
- Alert by Month — trend line of fraud-alert volume.
- Fraud Rate per Year — clustered column chart.

- Transaction Failure Reason — treemap breakdown.
- KPI summary cards and slicers for fraud-specific filtering.

Together, the two pages let executives track platform health at a glance while giving fraud analysts a drill-down workspace aligned with the device-channel and timing risk patterns identified in Sections 4–6.

8. Conclusion

This project delivered a validated, end-to-end analytics pipeline for a simulated UPI payments platform — from a referentially-intact MySQL schema, through documented data-quality assurance, exploratory analysis and formal statistical testing, to an operational Power BI dashboard. The core finding is that risk on this platform is not explained by simple, single-variable rules (device OS, channel, or amount) but concentrates in a specific behavioral pattern: Feature Phones transacting via QR Code, with fraud arriving in coordinated bursts rather than steady growth.